

AI ASSISTED CODING

ASSIGNMENT – 9.3

NAME : MANDALA.TEJASRI

ENROLLMENT NO : 2403A51171

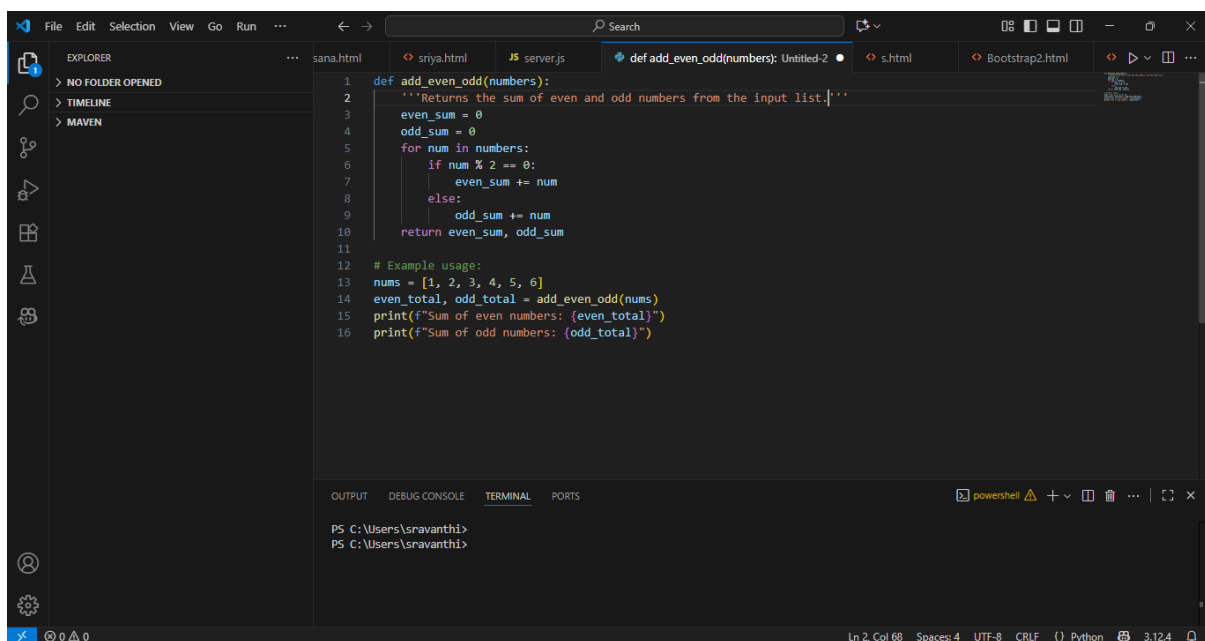
BATCH NO: 01

Task Description#1 Basic Docstring Generation

- Write python function to return sum of even and odd numbers in the given list.
- Incorporate manual docstring in code with Google Style
- Use an AI-assisted tool (e.g., Copilot, Cursor AI) to generate a docstring describing the function.
- Compare the AI-generated docstring with your manually written one.

Expected Outcome#1: Students understand how AI can produce function-level documentation.

MY CODE: Generate code for adding odd and even num separately.



The screenshot shows a code editor with a Python function `add_even_odd(numbers)` that calculates the sum of even and odd numbers in a list. The function includes a docstring, variable initialization, a loop with conditional logic, and a return statement. Below the function, there is an example usage section that defines a list `nums` and prints the results of the function call. The editor interface includes a sidebar with Explorer, Timeline, and Maven views, and a terminal at the bottom showing the current directory.

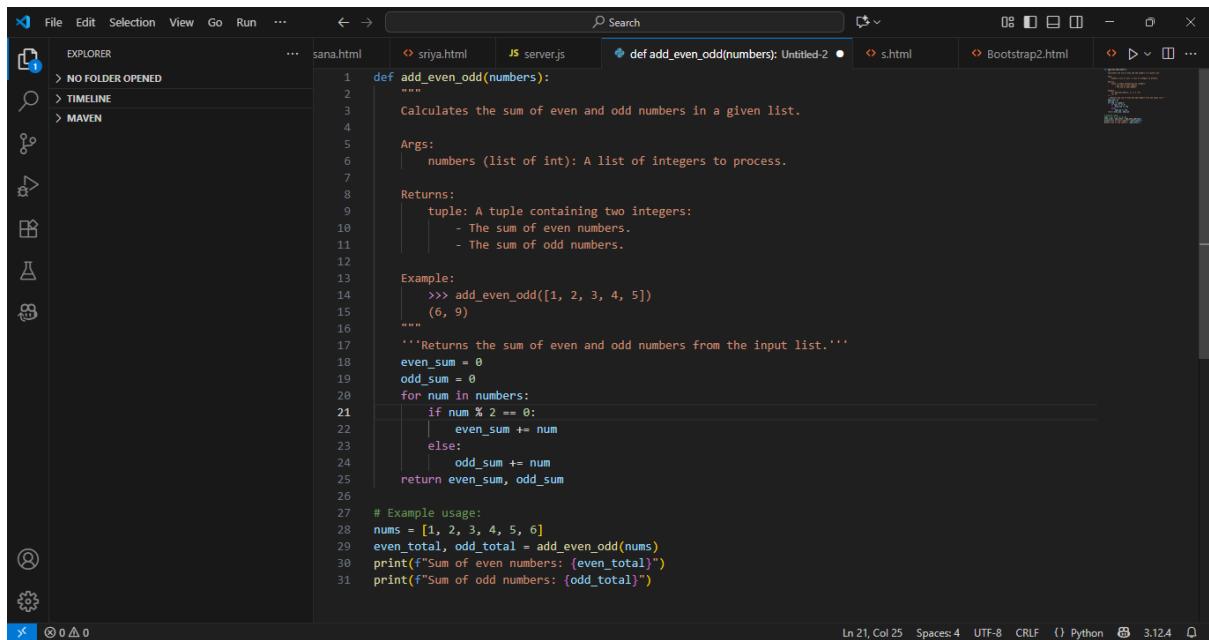
```
1 def add_even_odd(numbers):
2     """Returns the sum of even and odd numbers from the input list."""
3     even_sum = 0
4     odd_sum = 0
5     for num in numbers:
6         if num % 2 == 0:
7             even_sum += num
8         else:
9             odd_sum += num
10    return even_sum, odd_sum
11
12 # Example usage:
13 nums = [1, 2, 3, 4, 5, 6]
14 even_total, odd_total = add_even_odd(nums)
15 print(f"Sum of even numbers: {even_total}")
16 print(f"Sum of odd numbers: {odd_total}")
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\snavanthi>
PS C:\Users\snavanthi>

Ln 2, Col 68 Spaces: 4 UTF-8 CRLF Python 3.12.4

PROMPT: Generate doc for the code



The screenshot shows a VS Code editor window with a Python file named 'add_even_odd(numbers): Untitled-2'. The code defines a function 'add_even_odd' that calculates the sum of even and odd numbers in a given list. The function includes a docstring with a description, arguments, returns, and an example. The code is as follows:

```
1 def add_even_odd(numbers):
2     """
3     Calculates the sum of even and odd numbers in a given list.
4
5     Args:
6         numbers (list of int): A list of integers to process.
7
8     Returns:
9         tuple: A tuple containing two integers:
10             - The sum of even numbers.
11             - The sum of odd numbers.
12
13     Example:
14         >>> add_even_odd([1, 2, 3, 4, 5])
15         (6, 9)
16     """
17     '''Returns the sum of even and odd numbers from the input list.'''
18     even_sum = 0
19     odd_sum = 0
20     for num in numbers:
21         if num % 2 == 0:
22             even_sum += num
23         else:
24             odd_sum += num
25     return even_sum, odd_sum
26
27 # Example usage:
28 nums = [1, 2, 3, 4, 5, 6]
29 even_total, odd_total = add_even_odd(nums)
30 print(f"Sum of even numbers: {even_total}")
31 print(f"Sum of odd numbers: {odd_total}")
```

OBSERVATION:

Docstring by ai is far more better than mine code ,It just explained the code very clearly each line so even a non coders or non programmers can easily understand the code.

Task Description#2 Automatic Inline Comments

- **Write python program for sru_student class with attributes like name, roll no., hostel_status and fee_update method and display_details method.**
- **Write comments manually for each line/code block**
- **Ask an AI tool to add inline comments explaining each line/step.**
- **Compare the AI-generated comments with your manually written one.**

Expected Output#2: Students critically analyze AI-generated code comments

With my version:

Generate python program for sru_student class with attributes like name, roll no., hostel_status and fee_update method and display_details method and write comments like a human does

The screenshot shows a Google Colab notebook titled "Untitled46.ipynb". The code cell [2] contains the following Python code:

```
class SRUStudent:
    # Initialize the student with name, roll number, and hostel status
    def __init__(self, name, roll_no, hostel_status):
        self.name = name # Student's name
        self.roll_no = roll_no # Unique roll number
        self.hostel_status = hostel_status # True if staying in hostel, False otherwise
        self.fee_paid = False # Track if fee is updated

    # Method to update the fee status
    def fee_update(self, paid):
        """
        Update the fee status for the student.
        :param paid: Boolean, True if fee is paid, False otherwise
        """
        self.fee_paid = paid # Set fee status

    # Method to display student details
    def display_details(self):
        """
        Print all details of the student.
        """
        print(f"Name: {self.name}")
        print(f"Roll No.: {self.roll_no}")
        print(f"Hostel Status: {'Yes' if self.hostel_status else 'No'}")
```

The screenshot shows the same Google Colab notebook with two code cells. Cell [2] contains the same class definition as the previous image. Cell [3] contains the following code:

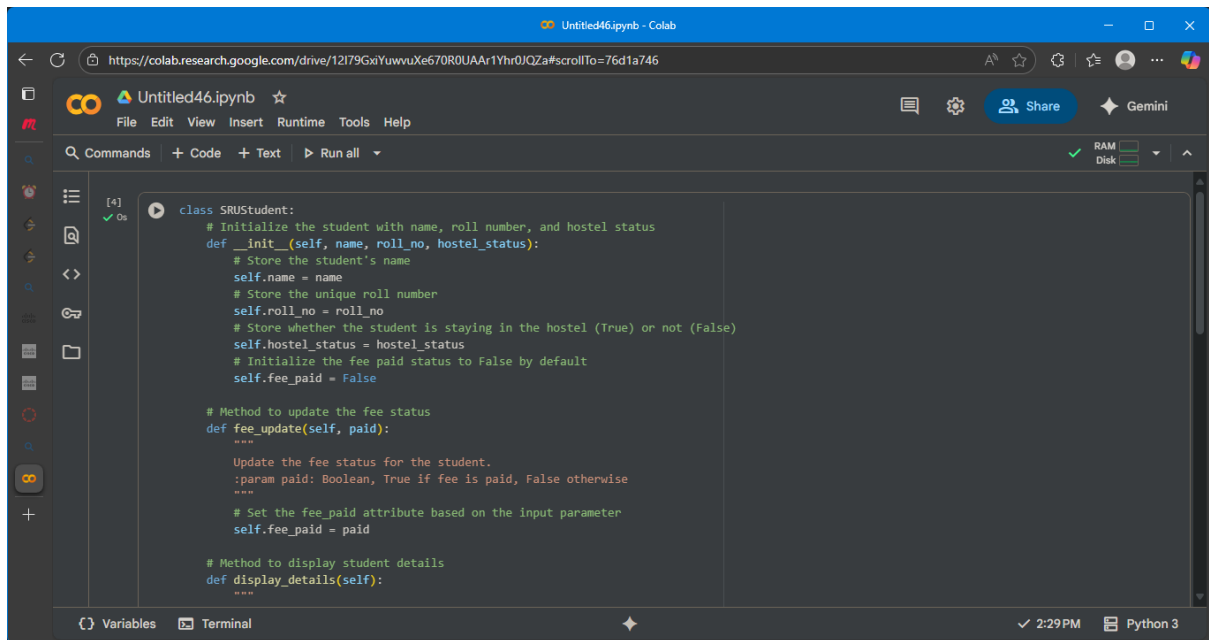
```
# Create a new SRUStudent object
student1 = SRUStudent("Alice", "SRU123", True)

# Display the details of the student
student1.display_details()
```

The output of cell [3] is displayed below the code:

```
Name: Alice
Roll No.: SRU123
Hostel Status: Yes
Fee Paid: No
```

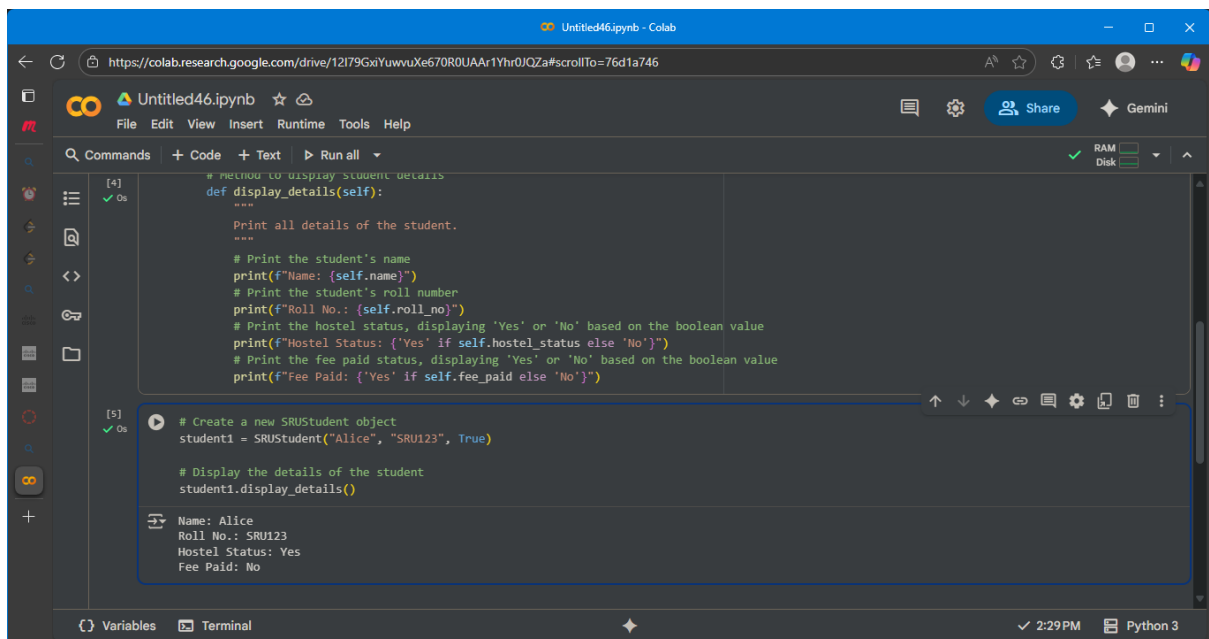
With ai version:



```
[4] ✓ Os
class SRUStudent:
    # Initialize the student with name, roll number, and hostel status
    def __init__(self, name, roll_no, hostel_status):
        # Store the student's name
        self.name = name
        # Store the unique roll number
        self.roll_no = roll_no
        # Store whether the student is staying in the hostel (True) or not (False)
        self.hostel_status = hostel_status
        # Initialize the fee paid status to False by default
        self.fee_paid = False

    # Method to update the fee status
    def fee_update(self, paid):
        """
        Update the fee status for the student.
        :param paid: Boolean, True if fee is paid, False otherwise
        """
        # Set the fee_paid attribute based on the input parameter
        self.fee_paid = paid

    # Method to display student details
    def display_details(self):
        """
```



```
[4] ✓ Os
    # Method to display student details
    def display_details(self):
        """
        Print all details of the student.
        """
        # Print the student's name
        print(f"Name: {self.name}")
        # Print the student's roll number
        print(f"Roll No.: {self.roll_no}")
        # Print the hostel status, displaying 'Yes' or 'No' based on the boolean value
        print(f"Hostel Status: {'Yes' if self.hostel_status else 'No'}")
        # Print the fee paid status, displaying 'Yes' or 'No' based on the boolean value
        print(f"Fee Paid: {'Yes' if self.fee_paid else 'No'}")

[5] ✓ Os
# Create a new SRUStudent object
student1 = SRUStudent("Alice", "SRU123", True)

# Display the details of the student
student1.display_details()

Name: Alice
Roll No.: SRU123
Hostel Status: Yes
Fee Paid: No
```

Observation:

**Comments by ai is far more better than mine code
It just explained the code very clearly each line so
even a non coders or non programmers can easily
understand the code.**

Task Description#3

- **Write a Python script with 3–4 functions (e.g., calculator: add, subtract, multiply, divide).**
- **Incorporate manual docstring in code with NumPy Style**
- **Use AI assistance to generate a module level docstring + individual function docstrings.**
- **Compare the AI-generated docstring with your manually written one**

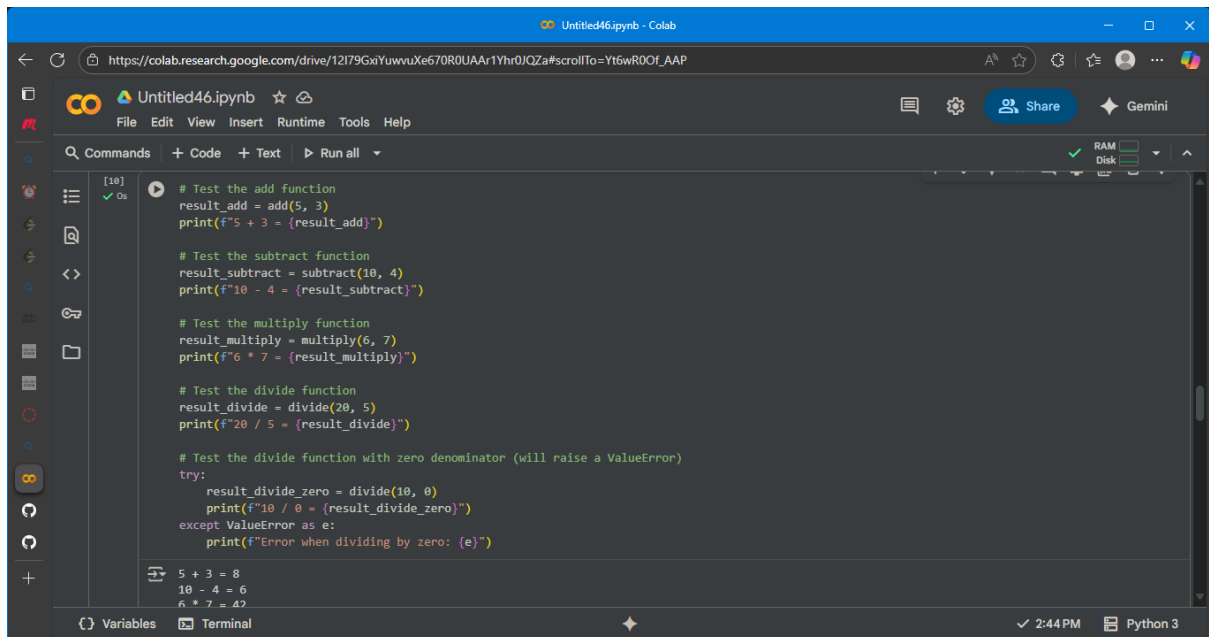
PROMPT:

Generate a python code to develop functions (e.g., calculator: add, subtract, multiply, divide). • Incorporate manual docstring in code with NumPy Style • Use AI assistance to generate a module-level docstring + individual function docstrings. • Compare the AI-generated docstring with your manually written one

Expected Output#3:

Students learn structured documentation for multi-function scripts

Push documentation whole workspace as .md file in GitHub Repository



The screenshot shows a Google Colab notebook titled "Untitled46.ipynb". The code defines four functions: `add`, `subtract`, `multiply`, and `divide`. It then tests each function with specific inputs and prints the results. A try-except block handles a `ValueError` when dividing by zero. The output shows the results of the calculations: `5 + 3 = 8`, `10 - 4 = 6`, `6 * 7 = 42`, and `20 / 5 = 4.0`. The status bar at the bottom indicates "Variables", "Terminal", "2:44 PM", and "Python 3".

```
[10] # Test the add function
result_add = add(5, 3)
print(f"5 + 3 = {result_add}")

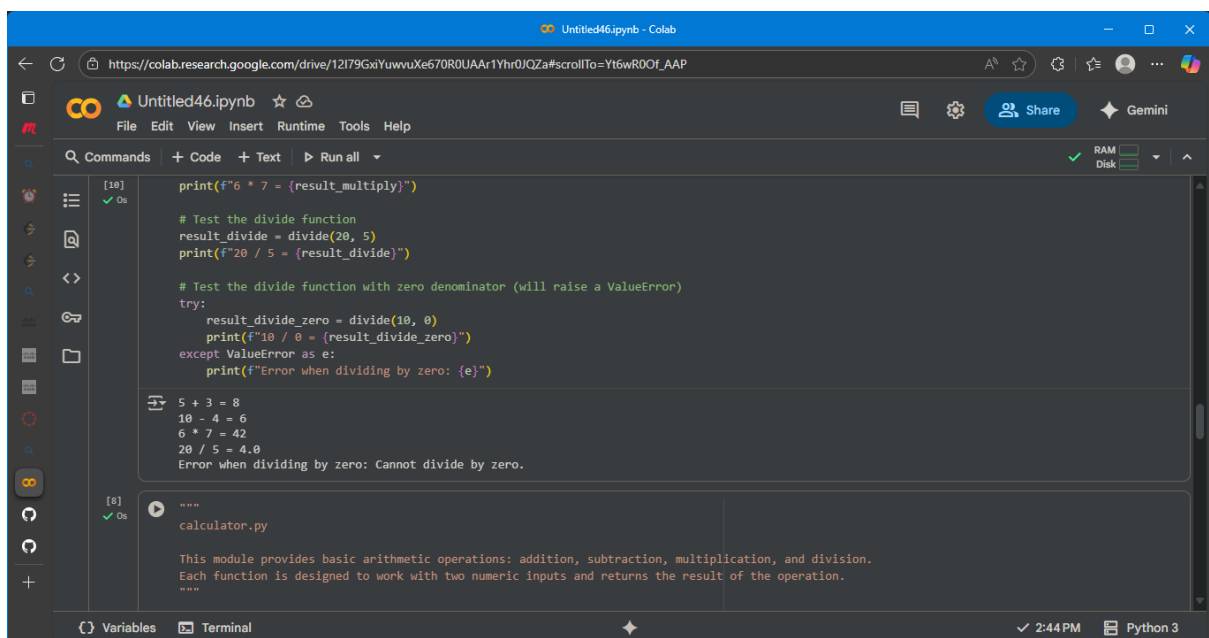
# Test the subtract function
result_subtract = subtract(10, 4)
print(f"10 - 4 = {result_subtract}")

# Test the multiply function
result_multiply = multiply(6, 7)
print(f"6 * 7 = {result_multiply}")

# Test the divide function
result_divide = divide(20, 5)
print(f"20 / 5 = {result_divide}")

# Test the divide function with zero denominator (will raise a ValueError)
try:
    result_divide_zero = divide(10, 0)
    print(f"10 / 0 = {result_divide_zero}")
except ValueError as e:
    print(f"Error when dividing by zero: {e}")

5 + 3 = 8
10 - 4 = 6
6 * 7 = 42
```



The screenshot shows the same Google Colab notebook, but now it includes a documentation block for the `calculator.py` module. The documentation describes the module's purpose and the functions it provides. The output shows the results of the calculations: `5 + 3 = 8`, `10 - 4 = 6`, `6 * 7 = 42`, and `20 / 5 = 4.0`. The status bar at the bottom indicates "Variables", "Terminal", "2:44 PM", and "Python 3".

```
[10] print(f"6 * 7 = {result_multiply}")

# Test the divide function
result_divide = divide(20, 5)
print(f"20 / 5 = {result_divide}")

# Test the divide function with zero denominator (will raise a ValueError)
try:
    result_divide_zero = divide(10, 0)
    print(f"10 / 0 = {result_divide_zero}")
except ValueError as e:
    print(f"Error when dividing by zero: {e}")

5 + 3 = 8
10 - 4 = 6
6 * 7 = 42
20 / 5 = 4.0
Error when dividing by zero: Cannot divide by zero.

[8] calculator.py

This module provides basic arithmetic operations: addition, subtraction, multiplication, and division.
Each function is designed to work with two numeric inputs and returns the result of the operation.
```

Observation:

Certainly! The selected code defines a simple calculator module with four basic arithmetic functions:

- **add(a, b):** This function takes two numbers, a and b, and returns their sum.
- **subtract(a, b):** This function takes two numbers, a and b, and returns the result of subtracting b from a.
- **multiply(a, b):** This function takes two numbers, a and b, and returns their product.
- **divide(a, b):** This function takes two numbers, a and b, and returns the result of dividing a by b. It also includes error handling to raise a `ValueError` if the denominator b is zero, as division by zero is undefined.

The code also includes docstrings for each function, explaining what it does, its parameters, what it returns, and any potential errors it might raise.